

QUICKBITE – INTELLIGENT FOOD DELIVERY SYSTEM

Database Systems (SWE2108)
Submitted to: Dr. Yaynshet Medhin

May 2026
Addis Ababa, Ethiopia

- Food delivery systems have evolved from simple telephone ordering services into intelligent, data-driven digital platforms.
- Rapid advancements in ICT, urbanization, and busy lifestyles have increased the demand for fast and convenient online food delivery services.
- Modern platforms must efficiently manage large volumes of data including customer information, orders, payments, restaurant operations, and delivery tracking.
- Poor database design can lead to delays, incorrect orders, data inconsistency, and reduced customer satisfaction.
- QuickBite solves these challenges through a hybrid database architecture that combines:
 - MySQL for structured transactional data
 - MongoDB for flexible and scalable document storage
- The system ensures:
 - High data integrity and reliability
 - Efficient real-time order processing
 - Scalability for growing user demand
 - Improved operational performance and customer experience
- QuickBite also integrates intelligent features such as delivery prediction, loyalty management, anomaly detection, and analytical reporting.
- The project demonstrates how modern hybrid database systems can support efficient, scalable, and real-world food delivery operations.

PROBLEM STATEMENT

- Despite the rapid growth of online food delivery platforms, many existing systems still face major challenges related to database management, scalability, and operational efficiency.
- Traditional systems often struggle to handle the increasing volume of customer transactions, delivery activities, and real-time operational data generated daily.
- Poor database design and weak integration between system components frequently result in:
 - Data inconsistency across multiple system modules
 - Data redundancy and duplicate records
 - Incorrect order processing and delayed deliveries
 - Reduced system reliability and customer satisfaction
- Many existing platforms also experience scalability limitations when handling large datasets and high user traffic.
- Slow data retrieval negatively impacts real-time order tracking, delivery coordination, and customer experience.
- Lack of advanced reporting and analytical capabilities limits business intelligence and strategic decision-making.
- To address these challenges, QuickBite proposes a hybrid database solution using MySQL and MongoDB to improve:
 - Data integrity and consistency
 - Query performance and scalability
 - Real-time operational efficiency
 - Reporting and analytical capabilities
 - Intelligent and data-driven decision making for modern food delivery services.

PROJECT OBJECTIVES

1

General Objective

- Design and implement an efficient and scalable food delivery database system using a hybrid MySQL and MongoDB approach.

Specific Objectives

- Analyze system requirements and data needs
- Design the ER diagram and logical schema
- Apply normalization up to BCNF
- Implement MySQL and MongoDB databases
- Create queries, reports, triggers, and procedures
- Improve scalability, integrity, and performance

2

Scope and Limitations

Scope

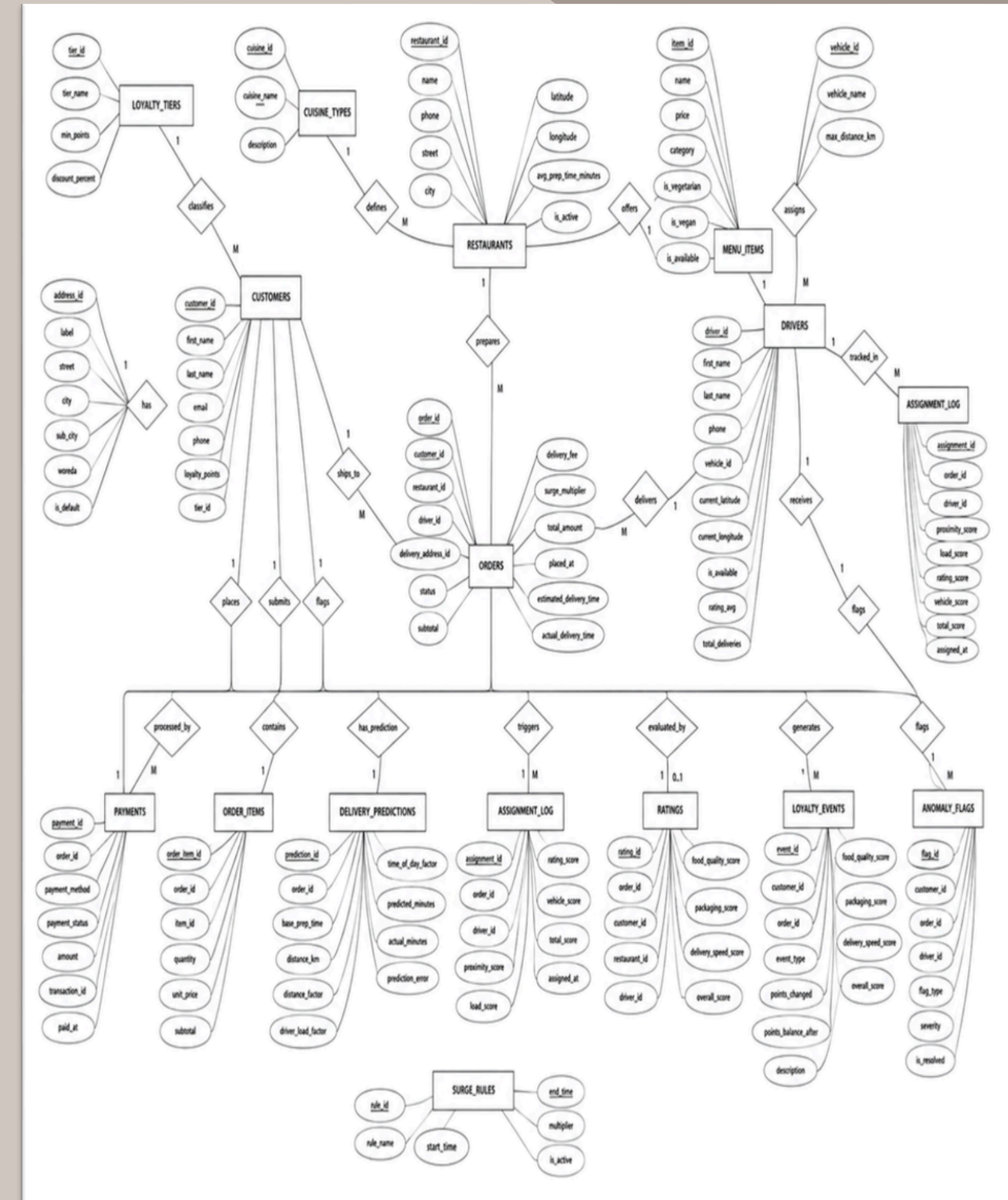
- Customer registration and profile management
- Restaurant and menu management
- Order tracking and delivery assignment
- Payment processing and reporting

Limitations

- No full web or mobile interface
- GPS tracking is simulated
- No real payment gateway integration
- Limited advanced analytics features

ER Diagram

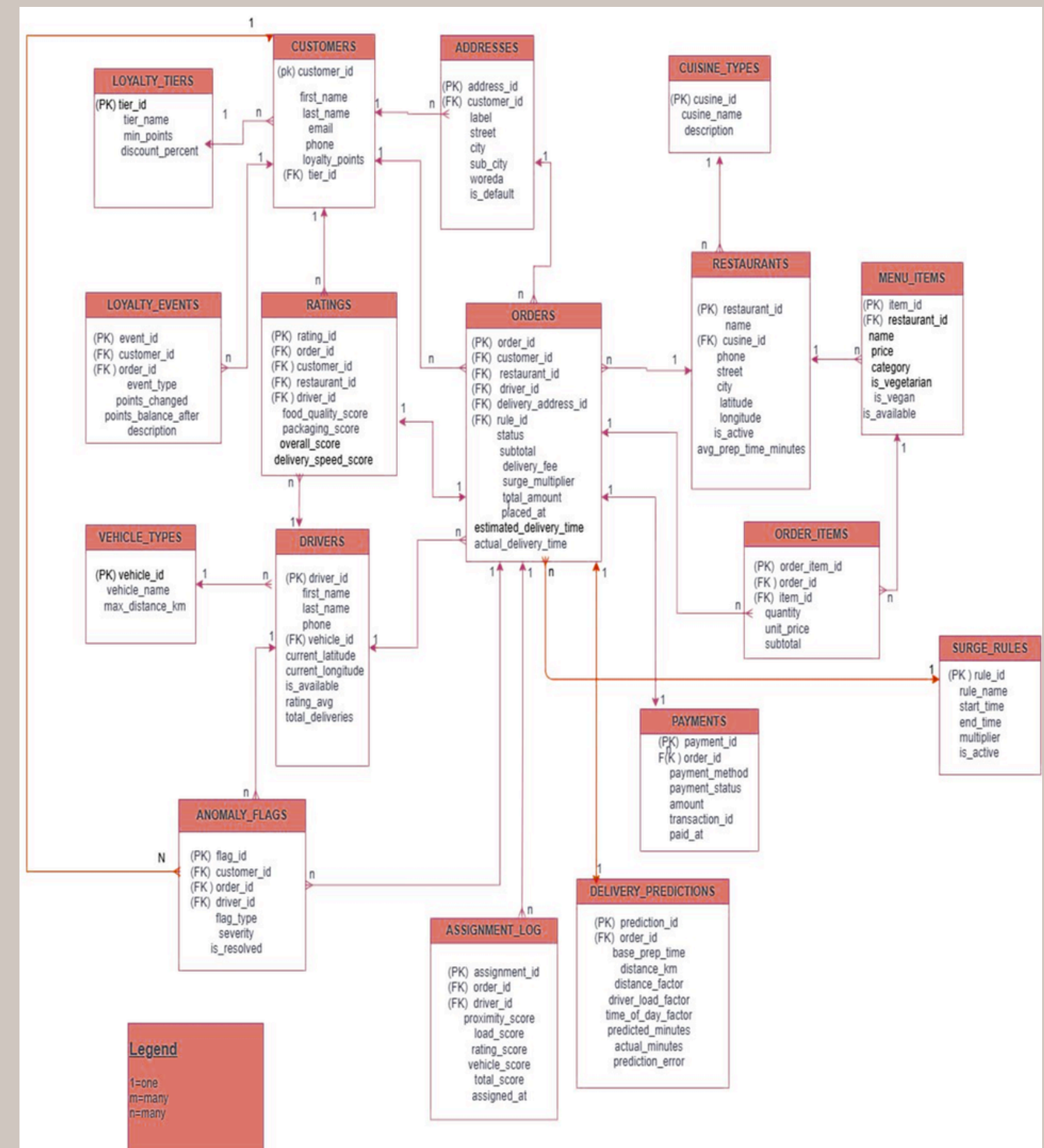
- The Entity-Relationship (ER) Diagram was designed using Draw.io to represent the complete database structure.
- The system contains 17 interconnected entities including Customers, Restaurants, Orders, Drivers, Menu Items, Payments, and Ratings.
- The ER model defines important one-to-many relationships such as:
 - Customers → Orders
 - Restaurants → Menu Items
 - Drivers → Deliveries
- The design ensures data integrity, reduced redundancy, and efficient system operations.
- The ER Diagram provides a clear visualization of the workflow and interactions within the QuickBite platform.



Database Design

Logical Schema Design

- The logical schema translates the conceptual ER model into implementable database tables and relationships.
- Each table was designed with clearly defined attributes, data types, primary keys, and foreign key constraints.
- Core transactional tables manage operational activities such as orders, payments, deliveries, and customer interactions.
- Supporting tables handle loyalty management, delivery prediction, assignment evaluation, and anomaly monitoring.
- The schema enables efficient data storage, faster retrieval, secure transaction handling, and accurate reporting.
- Proper structuring of relationships improves consistency and supports smooth integration between system components.



NORMALIZATION PROCESS

1

1NF (First Normal Form):

- All tables were structured with atomic values and unique primary keys, eliminating repeating groups and ensuring clean, organized data storage.

2NF (Second Normal Form):

- Partial dependencies were removed by ensuring every non-key attribute depends completely on the full primary key.

3NF (Third Normal Form):

- Transitive dependencies were eliminated so that non-key attributes depend only on their respective primary keys, improving consistency and reducing redundancy.

2

BCNF (Boyce-Codd Normal Form):

- The database was further refined so that every determinant functions as a candidate key, creating a more stable, efficient, and highly optimized schema.
- Key Outcomes:
 - Stronger data integrity and consistency
 - Reduced redundancy and storage waste
 - Faster updates and query execution
 - Improved scalability and database reliability for real-time operations

MYSQL TABLES OVERVIEW

- The system is implemented using 17 MySQL relational tables for a food delivery platform, structured into three main layers:

Business Core Tables:

- Customers, Orders, Restaurants, Menu Items, Drivers, Payments, Addresses, Deliveries, Ratings

→ Represent the operational backbone of the system, managing user activity, ordering workflow, and delivery processes

Intelligence Tables:

- delivery_predictions, anomaly_flags, recommendation_scores

→ Enable advanced system intelligence including predictive delivery times, anomaly detection, and personalized recommendations

Reference Tables:

- cuisine_types, loyalty_tiers, payment_methods, order_statuses

→ Maintain system-wide consistency by standardizing values and reducing data redundancy

Relationships:

- All entities are linked through one-to-many relational structures (e.g., Customer–Orders, Restaurant–Menu Items, Driver–Deliveries), ensuring structured data flow and referential integrity across the system

Constraints:

- Enforced using Primary Keys, Foreign Keys, NOT NULL, CHECK, and UNIQUE constraints

→ Guarantee data accuracy, validation, and consistency across all transactions

Design Quality:

- Fully normalized up to BCNF, minimizing redundancy, improving update efficiency, and ensuring scalability, performance optimization, and future system extensibility

VIEWS & STORED PROCEDURES

DATABASE VIEWS (4 VIEWS)

These views are pre-defined virtual tables used to simplify complex queries and provide real-time analytical insights without modifying the underlying data:

- Driver Performance Metrics
 - → Evaluates driver efficiency based on delivery time, completion rate, and reliability
- Order Summary Statistics
 - → Provides aggregated insights such as total orders, order status distribution, and order trends
- Prediction Accuracy Tracking
 - → Measures the accuracy of delivery time predictions by comparing predicted vs actual delivery times
- Restaurant Performance Analysis
 - → Analyzes restaurant performance based on order volume, ratings, and delivery outcomes

✓ These views improve data accessibility, reporting efficiency, and decision-making

STORED PROCEDURES (3 PROCEDURES)

Stored procedures are precompiled SQL routines used to automate complex system operations and improve performance:

- sp_assign_best_driver
 - → Automatically selects and assigns the most suitable driver based on performance score, distance, and availability
- sp_calculate_delivery_prediction
 - → Computes estimated delivery time using historical delivery patterns and system data
- sp_detect_anomalies
 - → Identifies unusual patterns such as delays, failed deliveries, or inconsistent system behavior

✓ These procedures ensure automation, consistency, and reduced manual processing

TRIGGERS FOR BUSINESS LOGIC AUTOMATION

- The system implements MySQL triggers to automatically enforce business rules and maintain real-time data consistency without manual intervention. These triggers ensure that key updates across customers, drivers, and ratings are always synchronized with system activity.

trg_after_order_delivered

→ Executes automatically when an order is marked as delivered

- Adds loyalty points to the customer account based on completed orders
- Increments the driver's total delivery count

✓ Ensures accurate reward tracking and driver activity monitoring without manual updates

trg_update_driver_rating

→ Activates after each new customer review

- Recalculates the driver's average rating in real-time
- Updates performance metrics used in driver assignment algorithms

✓ Maintains up-to-date driver evaluation for fair and efficient dispatching

trg_update_loyalty_tier

→ Monitors customer loyalty points continuously

- Automatically upgrades customer tier (Bronze → Silver → Gold) when thresholds are reached
- Reflects customer engagement and reward level instantly

✓ Supports dynamic loyalty management and customer retention strategies

Example System Result

After order completion:

- Customer loyalty points increased from 900 → 929

✓ Demonstrates real-time automation of rewards and seamless system updates

MONGODB COLLECTIONS

- The system integrates MongoDB collections alongside MySQL to handle flexible, semi-structured, and high-volume data efficiently. This supports a hybrid database architecture, combining relational integrity (MySQL) with schema flexibility (MongoDB) for improved scalability and performance.

Menu Catalog

→ Stores restaurant menu data with a flexible schema structure

- Supports varying item attributes (price, ingredients, availability)
- Handles modifiers (extra toppings, custom options)
- Allows seasonal or dynamic menu updates without schema redesign

✓ Enables high flexibility for restaurant-specific menu variations

Order Snapshots

→ Captures a complete static copy of an order at the time of placement

- Preserves full order details (items, prices, status, timestamps)
- Maintains historical accuracy even if original records are updated later

✓ Ensures reliable historical reporting and audit tracking

Customer Reviews

→ Stores unstructured and rich feedback data

- Includes ratings, textual comments, and image uploads

- Supports sentiment-based analysis for customer satisfaction insights

✓ Enables advanced analytics beyond structured relational constraints

Driver Activity Logs

→ Tracks real-time operational events of drivers

- Records location updates, delivery status changes, and timestamps
- Supports live monitoring and operational transparency

✓ Improves logistics tracking and system responsiveness

Purpose of MongoDB Integration

- Provides schema flexibility for dynamic and evolving data
- Handles high-volume, real-time, and unstructured information efficiently
- Works alongside MySQL in a hybrid database architecture

→ MySQL = structured relational data

→ MongoDB = flexible, scalable data storage

✓ Result: A scalable, efficient, and modern hybrid database system optimized for real-world food delivery applications

ANALYTICAL QUERIES OVERVIEW

- Seven advanced SQL queries power the system's analytics engine, enabling intelligent decision-making, performance tracking, and business optimization:

Delivery Prediction Accuracy

→ Compares actual vs. estimated delivery times to measure prediction reliability and improve forecasting accuracy

Driver Assignment Scoring

→ Ranks drivers based on proximity, rating, and availability to optimize delivery assignment decisions

Revenue Analysis with Surge Pricing

→ Uses window functions to analyze earnings across pricing tiers and evaluate revenue trends under surge pricing

Loyalty Report

→ Tracks customer loyalty points and tier progression (Bronze → Silver → Gold) to measure retention and engagement

Restaurant Rankings

→ Aggregates ratings and order volume to generate performance-based restaurant rankings

Anomaly Summary

→ Detects and summarizes suspicious or irregular orders for fraud detection and system monitoring

Recommendation Engine

→ Uses collaborative filtering to suggest restaurants based on similar user preferences and behavior patterns

✓ Overall, these queries transform raw transactional data into actionable insights, supporting prediction, optimization, fraud detection, and personalization within the system.

DELIVERY PREDICTION ACCURACY

The system evaluates the performance of its delivery time prediction model to ensure reliable and realistic estimated delivery times for users.

- The model achieves an average prediction accuracy of 81.67%, indicating strong alignment between predicted and actual delivery times
- It maintains a low average error margin of 9.6 minutes, making predictions practically reliable for real-world delivery operations

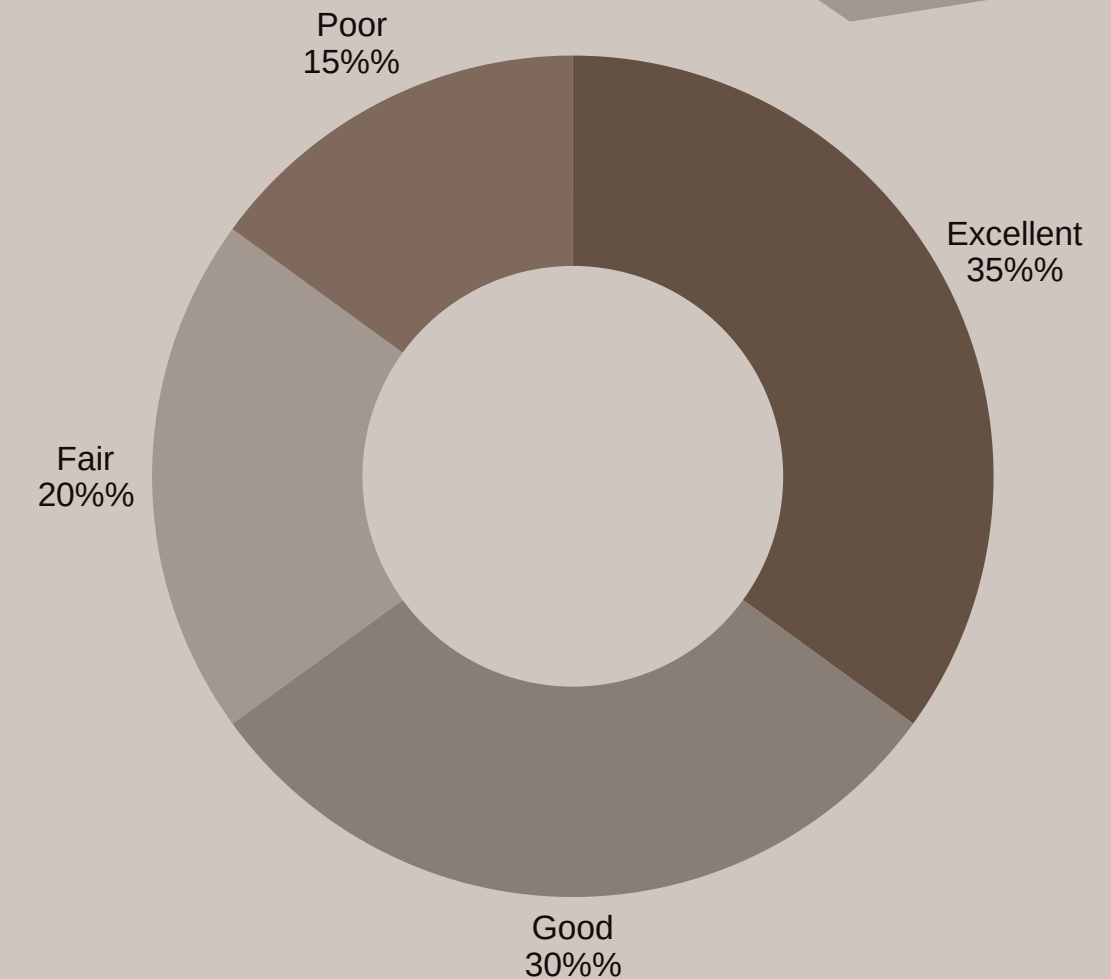
Performance Classification (Error-Based Tiers)

Prediction quality is categorized into four performance levels:

- Excellent: error within ≤ 5 minutes
- Good: error between 5 – 10 minutes
- Fair: error between 10 – 15 minutes
- Poor: error > 15 minutes

System Impact

- Enables continuous monitoring of prediction model performance
 - Helps identify areas for improving delivery estimation accuracy
 - Supports better customer satisfaction through realistic delivery time expectations
 - Strengthens logistics planning and operational efficiency
- ✓ Overall, this module ensures data-driven optimization of delivery predictions, improving both system reliability and user experience.



Source: AI-generated data. Replace or verify before use.

IMPLEMENTATION SUMMARY

- The system is implemented as a full-scale hybrid database architecture for a food delivery platform, integrating relational and non-relational technologies to ensure scalability, automation, and real-world data handling efficiency.
 - The system is implemented using 17 MySQL tables with full referential integrity, ensuring consistent relationships through primary and foreign keys.
 - Includes 4 database views for simplified reporting and real-time analytics.
 - Uses 3 stored procedures to automate key operations such as driver assignment, delivery prediction, and anomaly detection.
 - Implements 3 triggers to enforce business logic automatically (loyalty points updates, driver ratings, and tier upgrades).
 - Developed 7 advanced SQL analytical queries for insights into performance, prediction accuracy, revenue, rankings, and recommendations.
 - Integrated 4 MongoDB collections to handle flexible and semi-structured data (menus, reviews, logs, snapshots) within a hybrid DBMS.
 - Source code is managed using a GitHub repository for version control and collaboration.
- ### Key Technical Outcomes
- Strong mastery of ER modeling and relational schema design
 - Application of advanced database normalization up to BCNF, eliminating redundancy and update anomalies
 - Implementation of automation-driven database systems using triggers and stored procedures
 - Development of a hybrid DBMS architecture (MySQL + MongoDB) for structured and unstructured data handling
 - Practical exposure to real-world scalable system design and analytics-driven architecture
- ✓ Overall, the project delivers a production-style database ecosystem that combines theoretical database principles with enterprise-level implementation practices, ensuring performance, integrity, and scalability.

THANK YOU!!!

Questions? Visit our GitHub repository for source code

- To check out the repository go to :

<https://github.com/suzumi-dev/-AASTU-DB-Project-Group-4>

GROUP MEMBERS

1. AFOMIYA ASCHENAKI
2. AKSAH ATLAW
3. ARSEMA YESHIDNBER
4. BEREKET DERESSE
5. BETELHEM NURLACHEW
6. BETELHEM SOLOMON
7. MADING MAJOK

ID

- ETS 0111/17
- ETS 0121/17
- ETS 0199/17
- ETS 0262/17
- ETS 0284/17
- ETS 0286/17
- ETS 1876/17